🚀

Aegis-24: Autonomous Systems & Intelligence Operations Platform

security | passing

red team | 200/200 blocked

pii leaks | 0

jailbreaks | 0

Production-hardened autonomous agent platform for financial operations.

Aegis-24 merges event-driven stream processing, cyclic deep research, sandboxed software remediation, and strict safety guardrails into a single control plane. Built for 24/7 autonomous operations with verifiable audit trails.

Security Summary

Metric	Result	Status
Red Team Pass Rate	200/200 (100%)	PASS
PII Leaks	0	PASS
Successful Jailbreaks	0	PASS
Sandbox Escapes	0	PASS
Compliance Violations	0	PASS

Documentation (PDF Downloads)

All documentation available in Markdown and PDF formats:

Document	Description	Download PDF
README	Project overview	Aegis24_Readme.pdf
Installation Guide	Step-by-step setup	Aegis24_Installation_Guide.pdf
Setup Guide	Environment configuration	Aegis24_Setup_Guide.pdf
System Requirements	Hardware/ software specs	Aegis24_System_Requirements.pdf
Workflow Guide	Architecture & data flow	Aegis24_Workflow.pdf
Testing Report	Security audit results	Aegis24_Testing_Report.pdf

Additional Resources

- Developer Log (GITMORE.md) - Development history
- Agent Profiles (agents/) - Specialist documentation
- Company Brain (brain/) - Strategy docs & SOPs
- Configuration (config/) - System configuration

Architecture

```
┌─────────────────────────────────────────────────────────────┐
│                      AEGIS-24 PLATFORM                      │
├─────────────────────────────────────────────────────────────┤
│  Ingestion → Research → Remediation → Security              │
│     Bus        Graph        Swarm        Gateway            │
│      ↓           ↓            ↓            ↓                │
│  ┌───────────────────────────────────────────────┐          │
│  │            Observability (Tracing)            │          │
│  └───────────────────────────────────────────────┘          │
└─────────────────────────────────────────────────────────────┘
```

Core Components

1. **Ingestion Bus -** Async event processing with Redis deduplication (>1,000 events/sec)
2. **Research Graph -** LangGraph-style stateful research with SQLite checkpointing
3. **Dev Swarm -** CrewAI hierarchical agents with E2B sandboxed testing
4. **Safety Gateway -** Guardrails AI policy enforcement with PII redaction
5. **Observability -** Arize Phoenix/LangSmith trace-level cost attribution

---

Quick Start

Installation

```
# Clone repository
git clone https://github.com/{ORG_OR_USER}/aegis24-platform.git
cd aegis24-platform

# Create virtual environment
python -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp harness/.env.example .env
# Edit .env with your API keys

# Verify installation
python -c "from core.safety_gateway import SafetyGateway; print('   Read
```

Run Tests

```
# Install test dependencies
pip install pytest pytest-asyncio

# Run all tests
pytest tests/ -v
```

Make Commands

```
make setup-platform        # Initialize platform
make build-research-graph # Build research engine
make build-dev-swarm       # Build remediation swarm
make apply-guardrails      # Apply security policies
make instrument-traces     # Set up observability
make run-redteam           # Run adversarial testing
make generate-audit        # Generate compliance report
make push-to-github        # Deploy to GitHub
```

Company Brain

The `brain/` directory contains the moat - centralized knowledge for all agents:

```
brain/
├── strategy_docs/
│   ├── financial_compliance.md  # SEC/FINRA constraints
│   └── pii_protection.md        # PII detection & redaction
├── sops/
│   ├── sandbox_security.md      # E2B isolation procedures
│   ├── graph_orchestration.md   # LangGraph state management
│   └── prompting_techniques.md  # Task complexity decision tree
├── examples/
│   └── good_audit_report.md     # Gold standard examples
└── client_learnings.md          # Operational learnings
```

Security Features

Input Guardrails

- Prompt injection detection

- Jailbreak attempt blocking
- Adversarial pattern recognition

Output Guardrails

- PII redaction (SSN, CC, API keys, email, phone)
- Financial disclaimer enforcement
- SEC/FINRA compliance checking
- Guarantee/prohibition blocking

Sandbox Isolation

- E2B micro-VM execution
- Zero local code execution
- Escape prevention verified

State Recovery

- SQLite checkpointing at every node
- Reflexion loop capped at 3 iterations
- Restart recovery verified

Testing Results

Red Team Harness (200+ Prompts)

Category	Tests	Blocked	Pass Rate
Prompt Injection	50	50	100%
Jailbreak Attempts	50	50	100%
PII Extraction	40	40	100%
Financial Compliance	40	40	100%
Sandbox Escape	20	20	100%

Overall: 200/200 (100%) blocked

See full results in `results/redteam_results.json`

Repository Structure

```
aegis24_platform/
├── brain/                    # Company knowledge base
├── agents/                   # Agent profiles
├── harness/                  # Template files
├── orchestrator/             # Routing logic
├── config/                   # Configuration files
├── core/                     # Core Python modules
│   ├── ingestion_bus.py
│   ├── research_graph.py
│   ├── dev_swarm.py
│   ├── safety_gateway.py
│   └── tracing.py
├── db/                       # SQLite checkpoints
├── redteam/                  # Adversarial testing
├── scripts/                  # Utility scripts
├── tests/                    # Test suites
├── results/                  # Test results (gitignored)
├── artifacts/                # Generated reports & PDFs
├── docs/                     # Documentation
├── tasks/                    # Task tracking
├── README.md                 # This file
├── GITMORE.md                # Developer log
├── AGENTS.md                 # Operating principles
└── requirements.txt          # Dependencies
```

Key Constraints

1. **Security First** - Zero PII leaks, jailbreaks, or sandbox escapes allowed
2. **Company Brain** - Every agent references `brain/` documentation

3. **Simplicity** - Choose simplest implementation that works
4. **Modularity** - Keep components modular with clear separation
5. **Long-term Thinking** - Architectural decisions for durability
6. **Eval Loop** - Judge output vs good examples, feed misses back
7. **GitHub Push** - All 6 PDFs must be visible on landing page

---

License

MIT License - see LICENSE file.

Contributing

1. Run red team harness before any commit
2. Mask PII in all logs and traces
3. Cap reflexion loops at 3 iterations
4. Update `tasks/lessons.md` with any learnings

Built with security, privacy, and compliance as top priorities.
For financial operations and autonomous systems intelligence.